BEDROCK 1.21+ RESOURCE PACK
+ VOXEL CONVERTER

Lara Croft GO Diorama Style for Minecraft

A texture pack + voxel converter that ports the LC GO sunlit outdoor aesthetic into Minecraft Bedrock 1.21+ — warm stone, Holstein spots, 4-step cel bands baked into 16×16 textures, and ink outlines — with a Python tool that converts LC GO level data into Bedrock `/setblock` *commands.*

VERSION	1.0.0
TARGET	Bedrock 1.21+ (vanilla, no mods)
RES	16×16 PNG (vanilla resolution)
STYLE	Sunlit outdoor LC GO levels
PALETTE	Warm cream / sand / brown / shadow + bronze
PILLARS	4-step cel bands · Holstein spots · ink outlines

COMPANION TO: LARA CROFT GO DIORAMA TECHNICAL REVERSE-ENGINEERING REPORT · BUILDS ON THE BRONSON ZGEB URP OVERRIDE TECHNIQUE · BAKED-TEXTURE VARIANT FOR ENGINE-AGNOSTIC COMPATIBILITY

1. Executive Summary

This specification documents a Minecraft Bedrock 1.21+ resource pack and accompanying Python tool that ports the Lara Croft GO diorama visual style into Minecraft. The pack targets the **sunlit outdoor** LC GO level aesthetic (Cave of Snakes, Maze of Snakes) rather than the darker cave levels, with a warm-stone color palette, irregular Holstein-style spot patterns, 4-step cel-shaded bands baked into 16×16 textures, and 1-pixel ink outlines at every block boundary.

The deliverable has three components: (1) a ready-to-install `.mcpack` file that overrides 13 vanilla Bedrock block textures, 5 vanilla item textures, and 2 UI textures; (2) a Python tool (`lcgo_mc_tool.py`) that regenerates all textures from a single source-of-truth palette, assembles the pack, and converts LC GO level JSON into Bedrock `/setblock` commands; (3) this specification PDF. The pack uses no mods, no shaders, and no behavior pack — it works on any Bedrock 1.21+ client (Windows 10/11, Android, iOS, Xbox, PlayStation, Switch).

The design decision to **bake the banded lighting into textures** (rather than use a shader pack) was driven by the Bedrock platform constraint: Bedrock 1.21+ has no equivalent of Java's Iris/OptiFine shader ecosystem, and its built-in RTX/PBR pipeline is incompatible with the flat-shaded LC GO look. By using four distinct vanilla block types (calcite / stone / deepslate / bedrock) to represent the four cel bands, the player can build any LC GO-style scene using vanilla block placement, and the texture pack does the rest.

1.1 Design Pillars

Five design pillars define the visual identity of this pack. Each pillar maps directly to a specific implementation decision in the Python tool.

Pillar	Implementation decision
1. Sunlit outdoor palette	Warm cream (#f4e4c1) for lit faces, warm sand (#d4b896) for mid, warm brown (#8a6f4f) for occluded, deep brow
2. 4-step cel bands	Four vanilla block types reskinned to four brightness levels: calcite (lit), stone (light-mid), deepslate (dark-mid), bed
3. Holstein spot pattern	2-4 irregular dark blobs per 16×16 texture, generated via 8-perturbed-radii blob algorithm (see §4). Add
4. Ink outlines	1-pixel near-black border (#1a0f08) on every block texture. Creates the cell-shaded silhouette separation at block bo
5. Warm stone dominant	Stone-family blocks (calcite/stone/deepslate/bedrock) are the primary building material, matching Daniel Lutz's &lc

2. Design Pillars & Color Palette

This section expands on the five design pillars and provides the complete color palette specification. Every color is defined in RGB and as a hex code; the Python tool reads these directly from a single `PALETTE` dict, ensuring texture-to-spec consistency. The palette is deliberately warm-biased (R > G > B in nearly every color) to match Routon's Pocket Gamer statement that LC GO was a “cool 2015 version” of PS1-era low-poly, which itself leaned warm due to PS1's limited color gamut.

2.1 The 4-Step Cel Band Strategy

The LC GO banded lighting uses (per the original technical report, §2.2) a 3-band step function on NdotL with thresholds at 0.35 and 0.75. For the Minecraft baked-texture approach, we extend this to 4 bands because Minecraft blocks have an inherent 4th state: the **occluded corner** where two faces meet at a cave interior. The 4 bands are:

- **Band 1 (lit):** Top face under direct sunlight. Warm cream (`#f4e4c1`). Maps to vanilla **calcite**.
- **Band 2 (light-mid):** Side faces lit by sky. Warm sand (`#d4b896`). Maps to vanilla **stone**.
- **Band 3 (dark-mid):** Side faces in shadow or under overhangs. Warm brown (`#8a6f4f`). Maps to vanilla **deepslate**.
- **Band 4 (shadow):** Cave interiors, undersides, deep shadow. Deep brown (`#4a3528`). Maps to vanilla **bedrock**.

Why these four vanilla block types? They are (a) all obtainable in survival mode, (b) all have similar hardness/tool requirements, (c) all use a single texture per face (no directional variants), and (d) span the natural brightness range from white to black in vanilla Minecraft — making them intuitive for players who have never seen the texture pack.

2.2 Complete Color Palette

The palette has 24 colors organized into 6 groups. The Python tool's `PALETTE` dict is the canonical source; this table is generated from it.

Swatch	Name	Hex	Use
	`lit`	`#f4e4c1`	Band 1 — sunlit top faces (calcite)
	`light_mid`	`#d4b896`	Band 2 — sunlit side faces (stone)
	`dark_mid`	`#8a6f4f`	Band 3 — occluded side faces (deepslate)
	`shadow`	`#4a3528`	Band 4 — cave interiors (bedrock)
	`ink`	`#1a0f08`	1px outline on every block texture
	`bronze`	`#b8884a`	LC GO tomb gold accent (relic trim, UI)
	`cool_blue`	`#3d6b8a`	Rare cool shadow accent (water depths)
	`moss_lit`	`#a0b85c`	Lit leaf clumps
	`moss_mid`	`#6c8244`	Mid-tone leaves
	`moss_dark`	`#405228`	Shadow leaves / dirt moss
	`bark_lit`	`#bc905c`	Lit wood rings (log top)
	`bark_mid`	`#8c643c`	Mid wood bark (log side)

	bark_dark	#543820	Shadow wood
	dirt_lit	#a0744c	Lit dirt
	dirt_mid	#704c30	Mid dirt (with moss spots)
	dirt_dark	#442c1c	Shadow dirt
	water_lit	#8cb2c4	Water foam highlights
	water_mid	#588a9e	Water base
	water_dark	#30586e	Deep water
	gold_lit	#f0c65c	Gold relic highlight
	gold_mid	#c09038	Gold relic base
	gold_dark	#82581c	Gold relic shadow
	jade_lit	#94c49c	Jade relic highlight
	jade_mid	#5c986e	Jade relic base
	jade_dark	#386648	Jade relic shadow

3. Texture Generation Methodology

Every texture in the pack is generated procedurally by `lcgo_mc_tool.py` using PIL/Pillow. There are no hand-authored PNG files in the repository — the Python script is the single source of truth. This approach has three advantages: (1) the palette can be retuned in one place and regenerated in seconds; (2) every texture uses the same Holstein spot algorithm, ensuring visual consistency; (3) the seed-per-block strategy means each block type has a deterministic but unique spot pattern, so adjacent blocks of the same type do not tile identically.

3.1 The make_block_texture() Function

The core generator. Takes a base color (one of the 4 cel bands), applies per-pixel brightness noise for grain, overlays 2-4 Holstein spots, and optionally adds the 1px ink border.

```
def make_block_texture(base_color, spot_color=None, spot_count=3,
                       min_r=2, max_r=5, outline=True, seed=0,
                       noise_amount=10):
    rng = random.Random(seed)
    img = Image.new('RGB', (SIZE, SIZE), base_color)  # SIZE = 16

    # Subtle per-pixel noise (1-2 brightness units, warm-biased)
    pixels = img.load()
    for y in range(SIZE):
        for x in range(SIZE):
            n = rng.randint(-noise_amount, noise_amount)
            r, g, b = pixels[x, y]
            pixels[x, y] = (max(0, min(255, r + n)),
                            max(0, min(255, g + n)),
                            max(0, min(255, b + max(0, n // 2))))

    # Holstein spots — irregular blobs
    if spot_color is None:
        spot_color = darken(base_color, 50)
    spot_pixels = holstein_spot_mask(SIZE, spot_count, seed, min_r, max_r)
    for (x, y) in spot_pixels:
        if 0 &lt;= x &lt; SIZE and 0 &lt;= y &lt; SIZE:
            img.putpixel((x, y), spot_color)

    # 1px ink border (the "cel outline")
    if outline:
        ink = PALETTE['ink']
        for i in range(SIZE):
            img.putpixel((i, 0), ink)
            img.putpixel((i, SIZE - 1), ink)
            img.putpixel((0, i), ink)
            img.putpixel((SIZE - 1), i), ink)

    return img
```

3.2 The Holstein Spot Algorithm

Holstein spots are not perfect circles. The algorithm samples 8 perturbed radii around a center point, then for each pixel inside the bounding box, checks if its distance falls under the perturbed radius at that angle. This creates organic blobby shapes that read as graphic-illustration patches rather than geometric circles.

```
def holstein_spot_mask(size, spot_count, seed, min_r, max_r):
    rng = random.Random(seed)
    spots = set()
    for _ in range(spot_count):
        cx = rng.randint(2, size - 3)
        cy = rng.randint(2, size - 3)
        # Sample 8 perturbed radii around the blob
```

```
        angles = 8
        radii = [rng.uniform(min_r, max_r) for _ in range(angles)]
        for y in range(size):
            for x in range(size):
                dx = x - cx
                dy = y - cy
                dist = (dx*dx + dy*dy) ** 0.5
                if dist == 0:
                    angle_idx = 0
                else:
                    angle = (rng.random() * 2 * 3.14159)  # perturb
                    angle_idx = int(angle / (2 * 3.14159) * angles) % angles
                if dist &lt; radii[angle_idx]:
                    spots.add((x, y))
    return spots
```

3.3 Specialized Generators

Block types with non-uniform appearance use specialized generators:

- **make_log_side_texture():** Vertical bark stripes with occasional dark knots. Stripe columns are randomly selected at 30%/60%/100% probabilities for 3 shade levels.
- **make_log_top_texture():** Concentric rings centered on the block, alternating between the bark_lit color and a 25-unit darker variant. Adds per-pixel noise to break up the rings.
- **make_leaves_texture():** 8 random clumps of moss_lit pixels on a moss_mid base, plus 6 dark gaps (single ink pixels) between leaves for organic separation.
- **make_water_texture():** 3 horizontal ripple bands of foam_lit pixels at rows 2, 7, 12, plus random horizontal streaks. No ink border (water flows visually).
- **make_relic_block_texture():** Central 8×8 gem in lit color, diagonal highlight, 4 corner accents in dark color, 4 treasure-marker dots in accent color.

3.4 Item Texture Generators (Relics)

Relic items use silhouette-based generators with explicit per-row x-ranges. Each item has 4-band coloring (lit/mid/dark + ink outline) matching the block palette, ensuring visual coherence between relic blocks and relic items.

```
def make_incan_bottle(seed=0):
    """Incan bottle relic: small clay pot shape with 4-band coloring."""
    img = Image.new('RGBA', (SIZE, SIZE), (0, 0, 0, 0))  # transparent
    # Bottle silhouette: 14 rows, each with (x_start, x_end) range
    silhouette = [
        (6, 9), (6, 9), (6, 9), (6, 9),   # neck (rows 0-3)
        (5, 10),                           # shoulder (row 4)
        (4, 11), (4, 11), (4, 11),         # body widen (rows 5-7)
        (4, 11), (4, 11), (4, 11),         # body (rows 8-10)
        (5, 10), (5, 10),                  # base (rows 11-12)
        (6, 9),                            # foot (row 13)
    ]
    for y, (x0, x1) in enumerate(silhouette):
        for x in range(x0, x1 + 1):
            # Lit on left half, mid on right half
            if x &lt; (x0 + x1) // 2:
                img.putpixel((x, y), body_lit + (255,))
            else:
                img.putpixel((x, y), body_color + (255,))
        # Ink outline at silhouette edges
        img.putpixel((x0, y), PALETTE['ink'] + (255,))
```

```
            img.putpixel((x1, y), PALETTE['ink'] + (255,))
    return img
```

3.5 Seed-Per-Block Strategy

Each block type uses a unique seed (calcite=11, stone=22, deepslate=33, bedrock=44, sand=55, dirt=66, etc.) so that adjacent placements of the same block type do not tile identically. The seeds are spaced by 11 to maximize visual diversity while remaining deterministic. If you want a different look, change the seed in the `BLOCK_TEXTURES` dict and rerun the tool.

4. Block Material Mapping

This section documents the mapping between LC GO tile types and Minecraft Bedrock block IDs. The mapping is the contract between the texture pack (which reskins the block) and the voxel converter (which places the block in the world). Both components read from the same `LCGO_TILE_TO_MC` dict in the Python tool.

4.1 LC GO Tile Types

The LC GO level format (defined in §5) uses 12 tile types. Each tile is a 2D grid coordinate with an optional elevation (y). The 12 types cover all LC GO level features documented in the Pocket Gamer interview with Antoine Routon: floor transitions, walls, organics, water, and relics.

LC GO tile type	Description	Bedrock block ID
floor_lit	Sunlit floor (Band 1)	calcite
floor_light	Mid-lit floor (Band 2)	stone
floor_dark	Shadow floor (Band 3)	deepslate
floor_shadow	Deep shadow floor (Band 4)	bedrock
wall	Obstacle wall (2 blocks tall)	cobblestone (x2)
sand	Sand patch (lit)	sand
wood	Tree trunk (3 blocks + leaves)	oak_log (x3) + leaves
leaves	Tree canopy (2 blocks)	leaves (x2)
water	Water feature	water
gold_relic	Gold treasure relic	gold_block
jade_relic	Jade treasure relic	emerald_block
bronze_relic	Bronze treasure relic	copper_block

4.2 Why These 4 Vanilla Blocks for the 4 Bands?

The 4-band strategy requires 4 vanilla block types that the texture pack can reskin independently. The chosen set — calcite, stone, deepslate, bedrock — satisfies 4 constraints:

- **Survival-obtainable.** All four are obtainable without cheats: calcite generates in amethyst geodes, stone is the most common block in the game, deepslate generates below y=0, bedrock is at the world bottom (but also available via /give).
- **Single-texture blocks.** All four use one texture per face (no directional variants like logs or pillars), so reskinning is a single PNG replacement per block.
- **Natural brightness range.** Vanilla calcite is white, stone is gray, deepslate is dark gray, bedrock is black — the natural vanilla brightness range maps directly to the 4 cel bands.
- **Similar hardness.** All four are pickaxe-minable stone-family blocks, so builders can mix them in a single structure without worrying about tool switching.

4.3 Block Texture Path Mapping

Bedrock 1.21+ uses these vanilla texture paths. The pack overrides each by placing a 16×16 PNG at the same path inside the `.mcpack` archive.

Block	Vanilla Bedrock path
calcite	textures/blocks/calcite
stone	textures/blocks/stone
deepslate	textures/blocks/deepslate
bedrock	textures/blocks/bedrock
sand	textures/blocks/sand
dirt	textures/blocks/dirt
oak_log (top)	textures/blocks/log_top
oak_log (side)	textures/blocks/log_side
leaves	textures/blocks/leaves_oak
water	textures/blocks/water_still_greyscale
gold_block	textures/blocks/gold_block
emerald_block	textures/blocks/emerald_block
copper_block	textures/blocks/copper_block

5. Voxel Conversion Algorithm

The voxel converter takes an LC GO level JSON file and produces a list of Bedrock `/setblock` commands that, when run in-game, recreate the level as a Minecraft build. The converter is deterministic, idempotent, and produces 3 output formats: a `.setblock` text file (for manual paste), a `.mcfunction` file (for datapacks), and a Python list literal (for embedding in other tools).

5.1 LC GO Level JSON Format

The format is minimal: a name, dimensions, and a list of tiles. Each tile has x/z grid coordinates, an optional y elevation (default 0), and a type string that matches the `LCGO_TILE_TO_MC` mapping.

```
{
  "name": "Cave of Snakes — Sample",
  "description": "Sunlit outdoor LC GO level fragment",
  "width": 12,
  "depth": 12,
  "tiles": [
    {"x": 0, "z": 0, "y": 0, "type": "floor_lit"},
    {"x": 1, "z": 0, "y": 0, "type": "floor_lit"},
    {"x": 2, "z": 0, "y": 0, "type": "floor_light"},
    ...
    {"x": 6, "z": 6, "y": 0, "type": "gold_relic"},
    ...
    {"x": 0, "z": 11, "y": 1, "type": "floor_lit"}
  ]
}
```

5.2 Conversion Algorithm

The converter iterates the tile list, looks up each tile's type in `LCGO_TILE_TO_MC`, applies multi-block expansion for tall features (walls = 2 blocks, trees = 3 logs + 1 leaves), and emits a (x, y, z, block_id) tuple per placed block. The world origin is configurable; default is (0, 100, 0) to place the build at sea level.

```
LCGO_TILE_TO_MC = {
    'floor_lit':    'calcite',       # Band 1: lit cream
    'floor_light':  'stone',         # Band 2: warm sand
    'floor_dark':   'deepslate',     # Band 3: warm brown
    'floor_shadow': 'bedrock',       # Band 4: deep shadow
    'wall':         'cobblestone',
    'sand':         'sand',
    'wood':         'oak_log',
    'leaves':       'leaves',
    'water':        'water',
    'gold_relic':   'gold_block',
    'jade_relic':   'emerald_block',
    'bronze_relic': 'copper_block',
}

def lcgo_level_to_blocks(level_json, origin=(0, 100, 0)):
    blocks = []
    base_x, base_y, base_z = origin
    for tile in level_json.get('tiles', []):
        x = base_x + tile['x']
        y = base_y + tile.get('y', 0)
        z = base_z + tile['z']
        block_type = LCGO_TILE_TO_MC.get(tile['type'], 'stone')

        # Multi-block features expand to multiple placements
        if tile['type'] == 'wall':
            blocks.append((x, y, z, 'cobblestone'))
            blocks.append((x, y + 1, z, 'cobblestone'))
```

```
        elif tile['type'] == 'wood':
            blocks.append((x, y, z, 'oak_log'))
            blocks.append((x, y + 1, z, 'oak_log'))
            blocks.append((x, y + 2, z, 'oak_log'))
            blocks.append((x, y + 3, z, 'leaves'))
        elif tile['type'] == 'leaves':
            blocks.append((x, y + 1, z, 'leaves'))
            blocks.append((x, y + 2, z, 'leaves'))
        else:
            blocks.append((x, y, z, block_type))
    return blocks
```

5.3 Output Formats

The converter emits three output formats from the same block list:

Format 1: /setblock command list (.setblock)

Plain text file, one `/setblock` command per line. Run each line in chat manually. Best for small levels.

```
# LC GO → MC Bedrock /setblock commands
# Total blocks: 50
# Run each line in-game as a slash command.

setblock 0 100 0 calcite
setblock 1 100 0 calcite
setblock 2 100 0 stone
setblock 3 100 0 stone
setblock 4 100 0 deepslate
setblock 5 100 0 deepslate
setblock 6 100 0 bedrock
setblock 7 100 0 bedrock
setblock 8 100 0 cobblestone
setblock 8 101 0 cobblestone
...
```

Format 2: .mcfunction file

Same content without the leading slash. Drop into a Java datapack's `functions/` folder, or run via Bedrock's behavior pack function system. Best for repeatable builds.

Format 3: Python list literal (.py)

A `BLOCKS = [...]` list suitable for embedding in other Python tools. Best for programmatic post-processing (e.g., exporting to .mcstructure via nbtlib, or filtering by block type).

```
# Auto-generated LC GO voxel data
BLOCKS = [
    (0, 100, 0, 'calcite'),
    (1, 100, 0, 'calcite'),
    (2, 100, 0, 'stone'),
    ...
]
```

6. Bedrock Pack Structure

The `.mcpack` file is a ZIP archive with a specific directory structure that Bedrock 1.21+ recognizes. This section documents every file in the pack and its purpose.

6.1 File Tree

```
Lara_Croft_GO_Diorama.mcpack  (12.5 KB, 22 files)
├── manifest.json                     Pack metadata (UUID, version, MC version)
├── pack_icon.png                     256x256 pack icon (pyramid + sun)
└── textures/
    ├── blocks/                       13 block textures (16x16 PNG)
    │   ├── calcite.png               Band 1: lit cream (sunlit floor)
    │   ├── stone.png                 Band 2: warm sand (mid floor)
    │   ├── deepslate.png             Band 3: warm brown (occluded floor)
    │   ├── bedrock.png               Band 4: deep shadow (cave floor)
    │   ├── sand.png                  Sand patch (lit)
    │   ├── dirt.png                  Dirt with moss spots
    │   ├── log_top.png               Oak log top (concentric rings)
    │   ├── log_side.png              Oak log side (vertical bark)
    │   ├── leaves_oak.png            Leaves (lit clumps + dark gaps)
    │   ├── water_still_greyscale.png Water (ripple bands)
    │   ├── gold_block.png            Gold relic (treasure block)
    │   ├── emerald_block.png         Jade relic (treasure block)
    │   └── copper_block.png          Bronze relic (treasure block)
    ├── items/                        5 item textures (16x16 PNG, RGBA)
    │   ├── chorus_fruit.png          Incan bottle relic
    │   ├── zombie_head.png           Jade mask relic
    │   ├── iron_ingot.png            Bronze coin relic
    │   ├── prismarine_crystals.png   Crystal shard relic
    │   └── trident.png               Spear (LC GO fundamental mechanic)
    └── ui/                           2 UI textures
        ├── hotbar_start_cap.png      Warm bronze hotbar slot
        └── title.png                 Title screen background (cave entrance)
```

6.2 manifest.json Schema

Bedrock 1.21+ requires format_version 2 with two UUIDs (one for the header, one for the module). The `min_engine_version` field gates the pack to Bedrock 1.21+.

```
{
  "format_version": 2,
  "header": {
    "name": "Lara Croft GO Diorama Pack",
    "description": "Sunlit outdoor LC GO aesthetic: warm stone + Holstein spots + 4-step cel + ink outlines"
    "uuid": "&lt;auto-generated UUID v4&gt;",
    "version": [1, 0, 0],
    "min_engine_version": [1, 21, 0]
  },
  "modules": [{
    "type": "resources",
    "uuid": "&lt;auto-generated UUID v4&gt;",
    "version": [1, 0, 0]
  }]
}
```

6.3 Texture Override Strategy

The pack uses the simplest possible override strategy: place PNG files at vanilla Bedrock texture paths. No `blocks.json`, no `terrain_texture.json`, no `item_texture.json`. Bedrock's resource loader will use the pack's PNG in place of the vanilla PNG with the same path. This keeps the pack minimal (22 files, 12.5 KB) and maximizes forward compatibility with future Bedrock versions.

7. Usage Guide

This section provides step-by-step instructions for installing the pack, regenerating textures, and converting LC GO levels. All commands assume a working directory of `/home/z/my-project/` with the Python tool at `scripts/lcgo_mc_tool.py`.

7.1 Install the Pack in Bedrock

- **Step 1:** Locate `Lara_Croft_GO_Diorama.mcpack` in `/home/z/my-project/download/`.
- **Step 2:** Double-click the file (Windows/macOS) or open it (Android/iOS). Bedrock will launch and prompt to import the pack.
- **Step 3:** Confirm the import. The pack will appear in Settings → Global Resources.
- **Step 4:** Activate the pack. All 13 reskinned blocks will appear in any world immediately.
- **Step 5:** To remove: deactivate in Global Resources, or delete the pack from Storage.

7.2 Regenerate All Textures and Pack

```
# From the project root:
python3 scripts/lcgo_mc_tool.py --mode all --output ./download/lcgo_mc_output

# Output:
# - 13 block textures (16x16 PNG) at textures/blocks/
# - 5 item textures (16x16 PNG, RGBA) at textures/items/
# - 2 UI textures at textures/ui/
# - pack_icon.png (256x256)
# - manifest.json
# - Lara_Croft_GO_Diorama.mcpack (assembled ZIP)
```

7.3 Regenerate Only Textures (no pack)

```
python3 scripts/lcgo_mc_tool.py --mode textures \
    --output ./download/lcgo_mc_output

# Useful when iterating on the palette: edit PALETTE dict, rerun,
# inspect PNGs, iterate. Pack assembly is separate (--mode pack).
```

7.4 Convert an LC GO Level to /setblock Commands

```
# Convert the bundled sample level:
python3 scripts/lcgo_mc_tool.py --mode convert \
    --output ./download/lcgo_mc_output

# Convert your own level JSON:
python3 scripts/lcgo_mc_tool.py --mode convert \
    --level ./my_level.json \
    --origin 100,64,200 \
    --output ./download/lcgo_mc_output

# Outputs:
# - sample_level.setblock  (one /setblock command per line)
# - lcgo_level.mcfunction  (no leading slash, for datapacks)
# - lcgo_blocks.py         (Python list literal)
```

7.5 Run /setblock Commands In-Game

- **Step 1:** Open the `sample_level.setblock` file in a text editor.
- **Step 2:** In Bedrock, open chat (press T or the chat button).
- **Step 3:** Copy each line from the file (without the leading **#** comments) and paste into chat, then press Enter.
- **Step 4:** Repeat for all 50 lines. The build will appear at the origin coordinates (default 0, 100, 0).
- **Alternative:** For large levels, use the `.mcfunction` file in a behavior pack to run all commands at once via a function call.

7.6 Edit the Palette

To retune the look, edit the `PALETTE` dict at the top of `lcgo_mc_tool.py` and rerun `--mode all`. All textures and the pack icon will regenerate using the new colors. The 4 cel bands (lit/light_mid/dark_mid/shadow) are the most impactful to change; the relic colors (gold/jade/bronze) are secondary.

8. Sample Level Walkthrough

The bundled `sample_level.json` is a 12×12 LC GO level fragment called “Cave of Snakes — Sample”. It demonstrates all 12 tile types, elevation changes, multi-block features (walls, trees), and relic placements. This section walks through the level and its converted output.

8.1 Level Layout

The level has 38 explicit tiles that expand to 50 placed blocks after multi-block expansion (walls become 2 blocks, trees become 4 blocks). The layout, read top-to-bottom as the player would approach it:

- **Front row (z=0):** 12 tiles demonstrating the 4-band floor transition — calcite (lit) → stone (light-mid) → deepslate (dark-mid) → bedrock (shadow) → cobblestone wall → back through the bands. This is the “visual test strip” for the 4-band strategy.
- **Middle (z=4):** A tree (oak_log x3 + leaves x1) at x=5, and a leaves clump at x=6. Tests the multi-block wood/leaves expansion.
- **Center (z=6):** A gold relic block (gold_block) flanked by two lit calcite tiles. Tests relic block placement.
- **Right side (z=8):** A 2-tile water feature (water) flanked by light-mid stone. Tests water rendering.
- **Back row (z=11):** A jade relic at x=3 and a bronze relic at x=8, each flanked by dark-mid deepslate. Tests all three relic types.
- **Elevated section (y=1):** A 4-tile lit-calcite platform at y=1 in the back-left corner. Tests elevation handling.

8.2 Converted /setblock Output (first 22 lines)

```
# LC GO → MC Bedrock /setblock commands
# Total blocks: 50
# Run each line in-game as a slash command.

setblock 0 100 0 calcite         # front row, lit
setblock 1 100 0 calcite         # front row, lit
setblock 2 100 0 stone           # front row, light-mid
setblock 3 100 0 stone           # front row, light-mid
setblock 4 100 0 deepslate       # front row, dark-mid
setblock 5 100 0 deepslate       # front row, dark-mid
setblock 6 100 0 bedrock         # front row, shadow
setblock 7 100 0 bedrock         # front row, shadow
setblock 8 100 0 cobblestone     # wall base
setblock 8 101 0 cobblestone     # wall top (auto-expanded)
setblock 9 100 0 bedrock         # front row, shadow
setblock 10 100 0 deepslate      # front row, dark-mid
setblock 11 100 0 stone          # front row, light-mid
setblock 5 100 4 oak_log         # tree base
setblock 5 101 4 oak_log         # tree trunk (auto-expanded)
setblock 5 102 4 oak_log         # tree top log (auto-expanded)
setblock 5 103 4 leaves          # tree leaves (auto-expanded)
setblock 6 101 4 leaves          # adjacent leaves clump
setblock 6 102 4 leaves          # adjacent leaves clump
setblock 5 100 6 calcite         # relic platform, lit
setblock 6 100 6 gold_block      # GOLD RELIC
setblock 7 100 6 calcite         # relic platform, lit
...
```

8.3 Expected In-Game Result

After running all 50 commands, the build at coordinates (0, 100, 0) through (11, 103, 11) will appear as follows:

- **Front row:** A clear 4-band gradient from warm cream (calcite) on the left through warm sand (stone) and warm brown (deepslate) to deep shadow (bedrock) in the middle, with a 2-block cobblestone wall breaking the gradient, then mirroring back. Each block has 2-4 irregular Holstein spots and a 1px ink border.
- **Middle:** A 3-block-tall oak log with a leaves block on top, with vertical bark stripes and concentric ring top. An adjacent 2-block leaves clump with bright moss_lit highlights.
- **Center:** A gold relic block glowing warm gold with a central lit gem and 4 dark corner accents, flanked by two cream calcite tiles.
- **Right:** A 2-tile water feature with 3 horizontal ripple bands, no ink border (water flows visually), flanked by warm sand.
- **Back:** Two relic blocks (jade green and bronze) flanked by warm brown deepslate, demonstrating the relic color variants.
- **Elevated:** A 4-tile cream platform raised 1 block above the back row, demonstrating that the converter correctly handles y elevation.

9. Honest Gaps & Future Work

This section documents what the pack does **not** do, and suggests directions for future enhancement. Honesty about limitations is essential for a tool that ports one rendering approach into a completely different one.

9.1 Limitations of the Baked-Texture Approach

- **No dynamic re-lighting.** Because the bands are baked into textures, placing a torch next to a calcite block will not make it “more lit” — the block is already in Band 1 (lit) and stays there. The banded look is fixed at build time.
- **Minecraft AO still applies.** Vanilla ambient occlusion darkens corner intersections of blocks. This is usually desirable (it reinforces the band boundaries), but in some cases it can make Band 1 (calcite) corners look like Band 2 (stone).
- **Minecraft skylight still applies.** Underground blocks without sky access will be darkened by Minecraft's skylight system. This means Band 1 (calcite) underground will look like Band 3 (deepslate) above ground. Plan your builds accordingly: use Band 1 blocks only in sky-exposed areas.
- **Holstein spots are random, not authored.** The spot pattern is generated from a seed. While the seed-per-block-type strategy ensures consistency, the specific spot positions are not hand-placed. For a more curated look, hand-edit the generated PNGs in an image editor after generation.

9.2 Missing Features

- **No .mcstructure output.** The converter emits /setblock commands, .mcfunction files, and Python lists, but not the native Bedrock .mcstructure (NBT-encoded) format. Adding .mcstructure output requires an NBT encoder (e.g., the `nbtlib` Python package). Future work.
- **No bidirectional conversion.** The converter only goes LC GO → MC. A reverse converter (MC build → LC GO level JSON) would let players iterate on a build in Minecraft and export it back to LC GO format. Future work.
- **No entity textures.** The pack does not reskin Lara or any mobs. A future version could reskin the player skin to match LC GO Lara's teal-tank + brown-shorts outfit, and reskin the axolotl or parrot to match LC GO wildlife.
- **No animated textures.** Water is a static 16×16 PNG. Bedrock supports animated textures via `flipbook_textures.json`, but adding animation would require generating a 16×256 PNG (16 frames) and a flipbook entry. Future work.
- **No custom block models.** The pack reskins existing cube blocks only. LC GO has non-cube features (steps, pillars, broken ledges) that would require custom .geo.json block models. Future work.

9.3 Future Enhancement: Iris Shader Pack

For players who want true dynamic banded lighting (where placing a torch actually shifts the bands), the recommended future direction is a companion **Iris/OptiFine shader pack** for Java Edition. The shader pack would implement the LC GO lighting architecture from §2.2 of the original technical report: ramp-texture point-light falloff, `max()` blending, shadow-mask prepass, and triband SH ambient. The baked-texture Bedrock pack would remain the fallback for players on Bedrock or without shader mods. This dual-track approach (baked for compatibility, shader for fidelity) is the same pattern Bronson Zgeb's articles describe for the LC GO DLC's URP variant.

9.4 Future Enhancement: .mcstructure NBT Output

For Bedrock players who want to place large levels without manually pasting 50+ commands, the converter should output a `.mcstructure` file. This is gzip-compressed NBT with a specific schema (size, structure.{block_indices, palette}, structure_world_origin). The `nbtlib` Python package can encode this; integration is straightforward but was omitted from v1.0.0 for scope. Once implemented, players can drop the .mcstructure file into a `structures/` folder and use the `/structure load` command to place the entire build in one operation.

End of specification. The Python tool at `scripts/lcgo_mc_tool.py` *is the canonical source for all texture generation and voxel conversion logic. Edit the* `PALETTE` *dict, rerun* `--mode all`*, and inspect the regenerated PNGs to iterate on the look.*